PROGRAMAÇÃO FRONT END

JS

Prof. Esp. Dacio F. Machado

</a> BOM DIA ! <\a>

SUPORTE_
A FIRMA ESTÁ MONTANDO UM TIME PARA DISPUTAR UM GRANDE TORNEIO DE FUTEBOL CONTRA OUTRAS EMPRESAS.
E VAMOS PRECISAR DE VOCÊ!
PRA JOGAR NO ATAQUE?
PRA HACKEAR O VAR.
vidadesuporte.com.br

SUPORTE_
ENTÃO VOCÊ SE ATRASOU PRO TRABALHO PORQUE A SUA VOVOZINHA FICOU DOENTE?
ACHO MEIO DIFÍCIL ACREDITAR NISSO!
Por quê?
POR QUE VOCÊ É UMA INTELIGÊNCIA ARTIFICIAL!
Eu sou adotado.
vidadesuporte.com.br

OBJETIVO DA AULA

FAZER AS COISAS FUNCIONAREM

→ conhecer o maior número de scripts <script> em **JS**

Introdução a JavaScript

JavaScript é considerada por alguns desenvolvedores como a **linguagem de programação da Web**. Isso porque todos os **navegadores modernos**, como desktop/notebook, consoles, tablets e smartphones, possuem interpretadores **JavaScript**.

Comandos JS

Você conhecerá, agora, como os comandos JavaScript são escritos, o que chamamos de sintaxe da linguagem.

O mais simples é o console.log, que serve para exibir um texto, como você fez, ou exibir o conteúdo de uma variável, que é um espaço de memória reservado para armazenar informações.

```html
<!DOCTYPE html>
<html lang="en">

<head>
    <title>Document</title>
</head>

<body>

    <script>
        console.log("Hello world!")
    </script>
</body>

</html>
```

```
console.log("Meu primeiro programa JS")
```

```
console.log("Meu primeiro programa");
```

```
> console.log("Meu Primeiro programa: trabalhando com variáveis");

  const idade = 26;

  console.log(idade)
  Meu Primeiro programa: trabalhando com variáveis
  26
```

- **Exemplo de operadores aritméticos: (valorDaHora * horasTrabalhadas) + bônus.** Os operadores são (+, -, * e /).
- **Exemplo de operadores de igualdade e relacionais:**
  - == igualdade (exemplo: **variavelX == variavelY**).
  - != diferença (exemplo: **variavelX != variavelY**).
  - > maior que (exemplo: **variavelX > variavelY**).
  - < menor que (exemplo: **variavelX < variavelY**).
  - >= maior ou igual que (exemplo: **variavelX >= variavelY**).
  - <= menor ou igual que (exemplo: **variavelX <= variavelY**).
- **Exemplo de operadores lógicos:**
  - && e - quando duas comparações são verdadeiras (exemplo: **1==1 && 2==2**).
  - || ou - quando uma das comparações é verdadeira (exemplo: **1==1 || dia=="domingo"**).
  - ! negação, inverte um valor booleano (exemplo: **!(1==1)** será falso).

Comentários

A instrução mais básica que podemos ter em uma linguagem de programação são os comentários do código, que são utilizados para dar informações sobre o código na sequência. Na Figura 1, podemos conhecer a estrutura básica de comentários em JavaScript:

```
// Tudo que é escrito após barras duplas ou duas barras é um comentário
// que não é executado pelo interpretador JavaScript.
// Ele tem efeito somente naquela linha que foi incluído.

/*
    Para comentários em blocos de múltiplas linhas você deverá utilizar a
    barra e o asterisco para iniciar o bloco de comentário e na sequência
    o asterisco e a barra para fechar o bloco de comentários
*/
```

Tipos de dados em JS

Javascript não é uma linguagem fortemente tipada, ou seja, não precisamos especificar o tipo de dado tratado por aquela variável.

Ao contrário, possui tipagem dinâmica que reconhece o tipo de dado dentre os tipos possíveis

- **Números**: utilizado apenas um tipo de *NUmber* para inteiros e reais.
- **Strings**: *strings* são textos que podem estar entre aspas (“) ou apóstrofos (').
- **Valores booleanos**: variáveis booleanas só aceitam *true* (verdadeiro) ou *false* (falso).
- **null**: é um valor especial que significa que a variável não tem nenhum valor.
- **undefined**: uma variável declarada, mas não definida com valores.

Variáveis e Dados

Porém, ainda temos diferenças de escopo. Em JS podemos especificar qual o escopo e a restrição de acesso a aquela variável

```
1 // Utilizando a palavra reservada var
2 var nome // declara variável nome disponível em todo projeto.
3 // Utilizando a palavra reservada let
4 let nome // declara variável nome disponível somente dentro do código onde foi criada.
5 // Utilizando a palavra reservada const
6 const nome // declara constante nome com a diferença que seu valor não pode ser alterado.
```

Estruturas de Dados

- **Objetos**: um objeto é uma coleção de pares de nome/valor ou um texto com um mapa de valores. Exemplo: **var livro = { assunto: “JavaScript”, nPaginas: 300 };**. A forma com a qual você acessa essas propriedades são as seguintes: **livro.assunto** ou **livro[“nPaginas”] .**
- **Arrays**: também conhecidos como vetores, são uma outra forma de se trabalhar valores em JavaScript. Exemplo: **var turnos = [“manhã”, “tarde”, “noite”];**, em que se pode acessar as informações da seguinte forma: **turnos[0]**, que exibirá “manhã”. E, também, há funções específicas para esse tipo, como **turnos.lenght**, que retorna o tamanho do array.

Pilares da Orientação a Objetos

O **Paradigma de Orientação a Objetos** utiliza, em sua essência, os conceitos de **classes** e **objetos**, pois, quando se programa em uma linguagem orientada a objetos, você tenta resolver um **problema do mundo real** para o computador

O Paradigma da Orientação a Objetos traz, em seu conceito, quatro pilares essenciais:

POO

ABSTRAÇÃO

Representações do mundo real, ou seja, a partir de um objeto real do nosso dia a dia você tem que imaginar como esse objeto se comportará dentro de sua aplicação.

HERANÇA

Herança é uma maneira de reutilizar o código. Nela, uma nova classe é criada, absorvendo os membros de uma classe já existente, que são aprimorados com novas ou melhores capacidades.

POLIMORFISMO

Podemos criar métodos dentro de uma classe que reagem ou funcionam de forma diferente dependendo do objeto que é executado.

ENCAPSULAMENTO

O objetivo deste pilar é restringir a visibilidade. Você sabe que tem a propriedade ou o método, mas não precisa saber como ele faz, somente o que ele faz.

```
class Carro {
 constructor(marca, modelo, caracteristicas) {
   this.Marca = marca;
   this.Modelo = modelo;
   this.Caracteristicas = caracteristicas;
 }
 exibirDetalhes() {
   alert(`Marca: ${this.Marca} - Modelo: ${this.Modelo}`);
 }
}
const carro1 = new Carro("Ford", "Ka", ["Preto", 1.0, "2
portas"]);
carro2.exibirDetalhes();
```

APLICANDO

OBRIGADO

daciofmf@gmail.com